ResearchHub AI

Intelligent Research Paper Management and Analysis System using Agentic AI

Project Description:

The emergence of scientific publications in various research fields has caused great difficulty for scientists, students and academicians to discover, organize, analyse and gain meaningful information from the published research papers. Researchers typically have to perform manual search, download papers, create folders, read long papers, and compare the results of various studies from different academic databases. Time consuming, repetitive and less productive.

Addressing these challenges, the ResearchPilot AI is an intelligent research paper management and analysis platform that adopts the principles of Agentic AI. Users can upload their own research documents, access a search function for academic papers, sort papers into dedicated workspaces, and utilize AI-powered insights of research materials.

It's developed with the help of backend services in FastAPI, frontend in React and TypeScript, managing metadata in SQLite, and Groq's Llama 3.3 70B model for cutting-edge natural language understanding and analysis functions. Research papers are processed with embeddings so that you can retrieve semantically and interact with the content based on the context.

Scenario 1: Intelligent Research Discovery

Users can use an integrated search interface to search academic articles and obtain the desired publications by searching for them and other metadata like title, publications source, publication date and authors. Selected papers can be imported straight into project specific workspaces, thereby establishing an organized research environment for additional analysis.

Scenario 2: AI-driven research analysis and reports:

ResearchPilot AI offers several AI features like Paper Summarizer, Insights Generator, Literature Review Generator, Paper Comparison, and Research Paper Reviewer. These tools can identify the structures of literature and automatically create research outputs, which can save time in manual literature analysis.

Scenario 3: Research Management in the Workspace:

Users can have several workspaces to work on different research projects and can keep different paper collections for every project. This allows for effective organization of research content and maintaining project-specific context and document relationships.

Scenario 4: Context-aware Research Assistant:

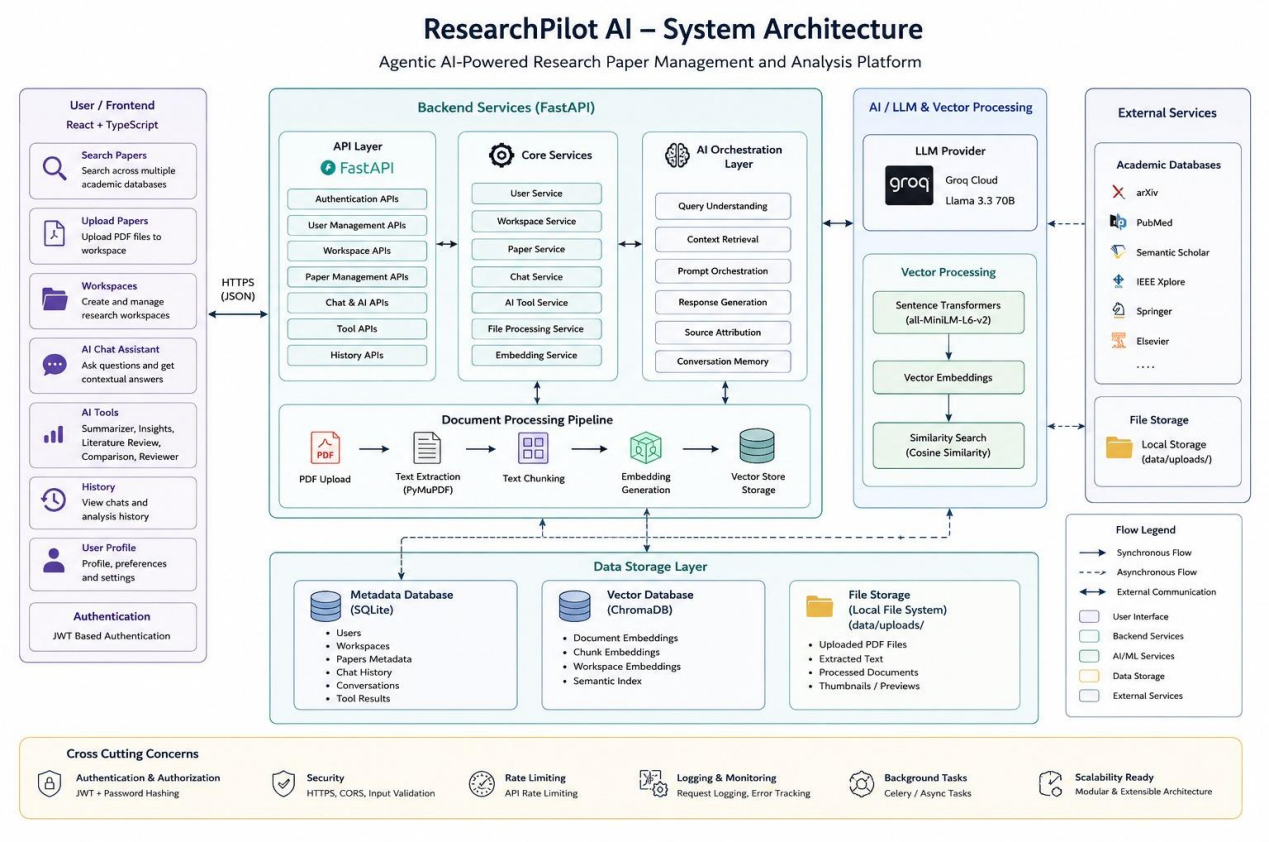
The AI Research Assistant features an AI-powered agent that is “workspace-aware” and can answer questions from uploaded research papers. The assistant uses vector embeddings and contextual retrieval to give evidence-based answers, summarize findings, pinpoint limitations and guide researchers along the entire literature review process.

Scenario 5: Research Document Review and Improvement:

ResearchPilot AI includes a Research Paper Reviewer that evaluates uploaded documents and provides structured feedback regarding strengths, missing sections, methodology quality, clarity, and overall document completeness. This assists users in improving project reports, research papers, and technical documentation before submission.

By combining intelligent paper discovery, semantic search, AI-powered analysis, workspace organization, and contextual research assistance, ResearchPilot AI significantly improves research productivity and enables users to focus on knowledge creation rather than manual information management.

Architecture:



Pre-requisites:

- FastAPI Framework Knowledge: FastAPI Documentation
- Groq API Familiarity: https://console.groq.com/
- HTML, CSS, and JavaScript Skills: W3Schools HTML/CSS/JavaScript Tutorials
- Python Programming Proficiency: Python Documentation
- Version Control with Git: Git Documentation
- Development Environment Setup: FastAPI Installation Guide
- React & TypeScript Development: React Document,Type Script Documentation

Key Features:

- Add or remove user from the system as required
.● Generate and assign JWT for user registration/Login.
- Research Paper Search from Academic Sources Upload PDF files and process documents.
- Multi-Workspace Research Management
- AI-Powered Research Assistant
- Paper Summarizer
- Insights Generator
- Literature Review Generator
- Paper Comparison Tool
- Research Paper Reviewer
- Vector-Based Semantic Search
- Conversation History Management
- Document Space for AI Report Storage
- Context-Aware Research Analysis

Step 1: Users sign up and log in by using a secure personal workspace authentication system using a JWT ensuring protection of the personal workspace, uploaded papers, and conversation history.Users register, and log in and out using a safe personal workspace authentication system based on a JWT, which protects the personal workspace, uploaded papers and conversation history.

Step 2: researchers query the integrated research paper search engine for search papers using keywords to search for papers. The backend fetches relevant papers with metadata like the title, the authors, the abstract, the source of publication and its date.

Step 3: users have the option to upload research papers in PDF format or import research papers they find in dedicated workspaces where they can be organized and analysed in the future.

Step 4: Uploaded documents are fed through the backend for text extraction, chunking, and generating vector embeddings for semantic retrieval.

Step 5: Researchers create several workspaces for papers, where they can manage different aspects of research and have different research contexts.

Step 6: The AI Research Assistant is based on Groq's Llama 3.3 70B model to use vector similarity search to index and fetch relevant information from papers, and produce context-appropriate answers.

Step 7: The users engage with an assistant and pose research-related inquiries, identify limitations, compare methods, pull out conclusions, and get evidence-based explanations from papers that have been uploaded.

STEP 8: AI-powered tools such as the Paper Summarizer, Insights Generator, Literature Review Generator, Paper Comparison, and Research Paper Reviewer can help researchers streamline and automate intricate research analysis.

STEP 9: Reports, analysis results, and AI-created outputs are automatically saved in the Document Space for easy access, review and project tracking

.

STEP 10: The platform, developed with React, TypeScript, FastAPI, SQLite, Vector Embeddings, and Groq Llama 3.3 70B, offers intelligent research support, semantic search, context-aware AI research analysis, and streamlined research workflows.

PRIOR KNOWLEDGE:

You must have prior knowledge of the following topics to complete this project:

Agentic AI and Groq API:

An in-depth look at AI agents, prompt engineering, RAG, and Groq's Llama 3.3 70B model for research analysis and support.

FastAPI:

Developing REST APIs with FastAPI, Processing documents, handling authentication, and integrating AI services.

Frontend Development (React, TypeScript, Tailwind CSS):

Create responsive user interface, state management, integration with APIs and modern frontend development with React, TypeScript & Tailwind CSS.

Vector Databases and Semantic Search:

Document Embedding for Contextual Search & AI Search.

SQLite Database Management:

Managing users, workspaces, papers, reports, and chat history.

PDF Processing

Extraction and analysis of the content of research papers for indexing and AI analysis.

PROJECT WORKFLOW

Milestone 1: Requirements Analysis and Project Setup:

Activity 1.1: Define project objectives and research workflow requirements
Activity 1.2: Set up frontend and backend project structure
Activity 1.3: Configure development environment and install required dependencies

Milestone 2: Database Design and Authentication System:

Activity 2.1: Design SQLite database schema for users, workspaces, papers, and chat history.
Activity 2.2: Implement JWT-based user authentication and authorization.
Activity 2.3: Develop user registration, login, and password recovery modules

Milestone 3: Research Paper Search and Management

Activity 3.1: Integrate academic paper search services (IEEE, arXiv, OpenAlex, Semantic Scholar, etc.)
Activity 3.2: Implement paper import and workspace management functionality
Activity 3.3: Develop paper metadata storage and document organization features

Milestone 4: PDF Processing and Vector Database Integration:

Activity 4.1: Implement PDF upload and document processing pipeline
Activity 4.2: Extract text from research papers and generate vector embeddings
Activity 4.3: Build semantic search and vector retrieval functionality

Milestone 5: AI Tools and Research Assistant Development:

Activity 5.1: Integrate Groq Llama 3.3 70B model for AI-powered analysis
Activity 5.2: Develop Paper Summarizer, Literature Review, Paper Comparison, and Insights Generator tools
Activity 5.3: Implement Research Paper Reviewer and Floating AI Assistan.
Activity 5.4: Build Research Chat with context-aware conversations and persistent chat history

Milestone 6: Frontend Development and User Interface Design

Activity 6.1: Develop Dashboard, Workspace, Search Papers, Upload PDF, and AI Tools pages
Activity 6.2: Implement responsive UI using React, TypeScript, and Tailwind CSS
Activity 6.3: Enhance user experience with modern UI components, chat interface, and workspace navigation

Milestone 7: Testing, Optimization, and Deployment

Activity 7.1: Perform functional and integration testing
Activity 7.2: Optimize API performance, vector search, and AI response workflows
Activity 7.3: Configure environment variables, CORS settings, and security policies
Activity 7.4: Deploy and validate the complete ResearchPilot AI platform

Milestone 1: Requirements Analysis and Project Setup

This milestone focused on defining the objectives, requirements, and overall architecture of the ResearchPilot AI platform. The development environment was configured, project structure was established, and all necessary dependencies were installed to create a strong foundation for future development. The primary objective was to prepare the system for research paper management, AI-powered analysis, semantic search, and workspace-based collaboration.

Activity 1.1: Define Project Objectives and Research Workflow Requirements

The initial phase involved identifying the key challenges faced by researchers when searching, organizing, and analyzing academic literature. Based on these requirements, the core functionalities of ResearchPilot AI were defined.

The main objectives included:

* Research paper discovery from multiple academic sources.
* Workspace-based research organization.
* PDF upload and document processing.
* AI-powered paper summarization and analysis.
* Semantic search using vector embeddings.
* Context-aware Research Chat.
* Persistent conversation history.
* Secure user authentication and data management.

These requirements served as the blueprint for designing the overall system architecture and development roadmap.

Activity 1.2: Set Up Frontend and Backend Project Structure:

The project was organized into separate frontend and backend modules to ensure scalability, maintainability, and modular development.

Project Structure:

```text
ResearchPilot-AI/
│
├── backend/
│   ├── app/
│   │   ├── routers/
│   │   ├── services/
│   │   ├── models/
│   │   ├── schemas/
│   │   └── core/
│   │
│   ├── data/
│   └── requirements.txt
│
```

```
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── context/
│   │   └── services/
│   │
│   └── package.json
│
└── README.md
```

The frontend was developed using React, TypeScript, and Tailwind CSS, while the backend was implemented using FastAPI and SQLite.

Activity1.3:ConfigureDevelopmentEnvironmentandInstallRequired Dependencies:

The development environment was configured by creating a Python virtual environment and installing all backend and frontend dependencies required by the application.

Backend Dependencies:

```txt
fastapi
uvicorn
sqlalchemy
python-dotenv
python-jose
passlib
bcrypt
chromadb
sentence-transformers
pypdf
groq
google-generativeai
httpx
python-multipart
```

Frontend Dependencies:

```bash
npm install react react-dom
npm install typescript
npm install tailwindcss
npm install axios
npm install react-router-dom
npm install lucide-react
```

Environment Setup:

```bash
# Create virtual environment

python -m venv venv

# Activate environment

venv\Scripts\activate

# Install backend packages

pip install -r requirements.txt

# Install frontend packages

cd frontend
npm install
```

Upon completion of this milestone, the ResearchPilot AI development environment was fully configured and ready for implementing authentication, workspace management, research paper search, vector databases, and AI-powered research tools..

Milestone 2: Database Design and Authentication System

This milestone focused on designing the core data management and security infrastructure of the ResearchPilot AI platform. The objective was to create a reliable database structure for storing users, workspaces, research papers, documents, AI-generated reports, and conversation history while ensuring secure user authentication and access control. This phase established the foundation upon which all other platform functionalities were built.

Activity 2.1: Design SQLite Database Schema

A relational database schema was designed using SQLite and SQLAlchemy ORM to manage the application's data efficiently.

The database consists of several interconnected entities including:

* Users
* Workspaces
* Research Papers
* Uploaded Documents
* Chat Sessions
* Chat Messages
* Generated Reports
* Security Logs

Relationships were established between users, workspaces, and documents to ensure proper

data organization and ownership tracking. This structure allows each user to maintain multiple research workspaces while keeping their research data isolated and secure.

Example database entities:

```python
# User
# Workspace
# Paper
# Document
# ChatSession
# ChatMessage
```

The database schema was designed to support scalability, maintain data integrity, and enable efficient retrieval of research information.

Activity 2.2: Implement JWT-Based Authentication and Authorization

A secure authentication system was implemented using JSON Web Tokens (JWT). This mechanism enables stateless authentication, allowing users to access protected resources without maintaining server-side sessions.

The authentication process includes:

1. User submits login credentials.
2. Credentials are verified against stored records.
3. A JWT access token is generated.
4. The token is returned to the client.
5. Protected API endpoints validate the token before granting access.

```python
from jose import jwt

access_token = create_access_token(
   data={"sub": user.email}
)
```

JWT authentication ensures that only authorized users can access workspaces, uploaded research papers, AI tools, and conversation history.

Activity 2.3: Develop User Registration, Login, and Password Recovery Modules

A complete user management system was developed to support account creation, authentication, and recovery operations.

The registration module allows new users to create accounts securely. Passwords are encrypted using hashing algorithms before being stored in the database, ensuring that raw passwords are never saved.

The login module validates user credentials and issues secure JWT access tokens for authenticated sessions.

Additional password recovery functionality was implemented to allow users to reset forgotten passwords securely through verification workflows.

Main authentication features include:

* User Registration
* Secure Login
* Password Hashing
* JWT Token Generation
* Password Reset
* Protected Routes
* Session Validation

These features collectively provide a secure foundation for managing user identities and protecting sensitive research data within the ResearchPilot AI platform.

By the completion of this milestone, the platform successfully established a robust database architecture and a secure authentication framework capable of supporting multiple users, workspaces, and AI-powered research operations.

Milestone 3: Research Paper Search and Management

This milestone focused on developing the research paper discovery, retrieval, and management infrastructure of the ResearchPilot AI platform. It involved integrating multiple academic databases, implementing workspace-based paper organization, and creating APIs for managing research documents. The objective was to provide researchers with a centralized platform for discovering, collecting, and organizing academic literature from various scholarly sources.

Activity 3.1: Integrate Academic Paper Search Services

This module enables researchers to search across multiple academic databases through a unified interface. The system integrates services such as IEEE Xplore, arXiv, Semantic Scholar, OpenAlex, CrossRef, PubMed, ACM Digital Library, and DOAJ.

The search API receives user queries, retrieves papers from multiple sources, removes duplicate entries, and ranks results based on relevance. The returned results include paper metadata such as title, authors, publication year, abstract, source, and citation information.

```python
# backend/app/routers/papers.py

@router.get("/search")
async def search_papers(
    query: str,
    current_user: User = Depends(get_current_user)
):
    results = await hybrid_search(query)
```

```
    return {
        "papers": results
    }
```

This functionality allows researchers to discover relevant academic literature without manually visiting multiple research repositories.

Activity 3.2: Implement Paper Import and Workspace Management

After discovering relevant papers, users can import them directly into their research workspaces. Workspaces serve as dedicated environments for organizing papers according to research topics, projects, or domains.

The import functionality stores paper metadata in the database and associates the selected paper with a specific workspace.

```python
# backend/app/routers/workspaces.py

@router.post("/{workspace_id}/papers/import")
async def import_paper(
    workspace_id: int,
    paper: PaperImport,
    current_user: User = Depends(get_current_user)
):
    imported_paper = save_paper(
        workspace_id,
        paper
    )

    return {
        "message": "Paper imported successfully"
    }
```

This workspace-centric approach enables researchers to maintain separate collections of literature for different projects while ensuring efficient organization and retrieval.

Activity 3.3: Develop Paper Metadata Storage and Document Organization

This module manages the storage and organization of research paper metadata within the platform. Information such as title, authors, abstract, publication date, source database, and workspace association is stored in the SQLite database.

The backend maintains relationships between users, workspaces, and imported papers, ensuring that research data remains properly organized and isolated between users.

```python
# backend/app/models/paper.py

class Paper(Base):
  __tablename__ = "papers"

  id = Column(Integer, primary_key=True)
  title = Column(String)
  authors = Column(Text)
  abstract = Column(Text)
  source = Column(String)
  workspace_id = Column(Integer)
```

The metadata management system enables efficient paper retrieval, supports AI-powered analysis workflows, and provides the foundation for semantic search, document processing, and research assistant functionalities implemented in later milestones.

By the completion of this milestone, ResearchPilot AI successfully provided a centralized research paper management environment capable of discovering, importing, organizing, and maintaining large collections of academic literature.

Milestone 4: Frontend Development with React and TypeScript

This milestone focused on developing a modern, responsive, and user-friendly frontend for the ResearchPilot AI platform using React, TypeScript, and Tailwind CSS. The frontend acts as the primary interaction layer between researchers and the AI-powered backend services. It involved building dynamic user interfaces, integrating REST APIs, managing application state, and creating an intuitive research workflow. The goal was to provide a seamless user experience while ensuring responsiveness, scalability, and maintainability.

Activity 4.1: Develop Authentication and User Management Interfaces

Authentication pages were developed to provide secure access to the platform. These interfaces include User Registration, Login, Forgot Password, OTP Verification, and Password Reset functionality.

The frontend communicates with FastAPI authentication APIs and securely stores JWT access tokens for authenticated sessions.

```tsx
```

```
// frontend/src/pages/Login.tsx



const handleLogin = async () => {
  const response = await api.post("/auth/login", {
    email,
    password,
  });

  localStorage.setItem(
    "token",
    response.data.access_token
  );
};
```

These interfaces ensure secure access to research workspaces, uploaded documents, AI tools, and chat history.

Activity 4.2: Develop Core Research Management Pages

Several core pages were developed to support the complete research workflow.

The implemented pages include:

* Dashboard
* Search Papers
* Workspace Management
* Upload PDF
* Research Chat
* AI Tools Hub
* Document Space

These pages allow researchers to search academic literature, manage workspaces, upload research papers, perform AI-powered analysis, and interact with the Research Assistant through a unified interface.

Activity 4.3: Implement Responsive UI and User Experience Enhancements

The user interface was built using Tailwind CSS with a modern dark-theme design focused on readability and productivity.

Major UI enhancements include:

* Responsive sidebar navigation
* Workspace switcher
* Interactive research chat interface
* AI-generated report viewer

* Floating AI Assistant
* Persistent chat history
* Professional dashboard cards
* Smooth transitions and animations

```tsx
// Example Navigation Component

<Sidebar>
 <Dashboard />
 <SearchPapers />
 <Workspace />
 <ResearchChat />
 <AITools />
</Sidebar>
```

Special attention was given to user experience by maintaining consistent layouts, intuitive navigation, fast page rendering, and seamless integration with backend services.

By the completion of this milestone, ResearchPilot AI successfully delivered a modern, responsive, and feature-rich frontend capable of supporting advanced AI-powered research workflows across multiple devices and screen sizes.

Milestone 5: AI Tools and Research Assistant Development

This milestone focused on integrating advanced AI capabilities into the ResearchPilot AI platform. It involved implementing Retrieval-Augmented Generation (RAG), semantic document retrieval, AI-powered research analysis tools, and a context-aware Research Assistant. The objective was to automate complex research tasks such as summarization, literature review generation, paper comparison, insights extraction, and conversational research assistance while maintaining accuracy through document-grounded responses.

Activity 5.1: Develop AI-Powered Research Analysis Functions

The Research Assistant was designed to analyze uploaded research papers and generate context-aware responses using the Groq Llama 3.3 70B model. Instead of relying solely on user prompts, the system retrieves relevant document chunks from the vector database and supplies them as context to the language model.

The retrieval process involves:

1. Converting user queries into vector embeddings.
2. Searching the Chroma Vector Database for relevant document chunks.
3. Constructing a contextual prompt using retrieved research content.
4. Sending the enriched context to the LLM.
5. Generating accurate research-focused responses.

```python id="rg8k12"
# backend/app/services/vector_service.py

def retrieve_relevant_chunks(query):
    query_embedding = generate_embedding(query)

    results = vector_store.similarity_search(
        query_embedding,
        top_k=5
    )

    return results
```

```python id="lm9d53"
# backend/app/services/llm_service.py

def generate_response(context, query):

    messages = [
        {
            "role": "system",
            "content": "You are an expert research assistant."
        },
        {
            "role": "user",
            "content": f"Context: {context}\nQuestion: {query}"
        }
    ]

    response = client.chat.completions.create(
        messages=messages,
        model="llama-3.3-70b-versatile"
    )

    return response.choices[0].message.content
```

This architecture enables the Research Assistant to generate responses grounded in uploaded research papers rather than relying entirely on model knowledge.

Activity 5.2: Develop AI Research Analysis Tools

Several AI-powered tools were implemented to assist researchers in analyzing and understanding academic literature.

The developed tools include:

* Paper Summarizer
* Literature Review Generator
* Paper Comparison Tool
* Insights Generator
* Research Paper Reviewer

These tools process selected papers and generate structured outputs such as summaries, comparative analyses, literature surveys, strengths and weaknesses, research gaps, and future directions.

```python id="tr5a81"
# backend/app/routers/tools.py

@router.post("/paper-summary")
async def summarize_paper():
    pass

@router.post("/literature-review")
async def generate_literature_review():
    pass

@router.post("/paper-comparison")
async def compare_papers():
    pass
```

The tools significantly reduce manual reading effort and accelerate research workflows.

Activity 5.3: Implement Context-Aware Research Chat and Persistent History

A dedicated Research Chat module was developed to provide conversational interaction with uploaded research papers. The system maintains chat sessions, stores conversation history, and preserves workspace-specific context.

The Research Chat workflow includes:

* Workspace-aware conversations
* Persistent chat history
* Multiple chat sessions
* Chat session management
* Context retention across interactions
* Semantic retrieval before response generation

```python id="cx4m92"
# backend/app/routers/chat.py

@router.post("/chat/session/{session_id}/message")
async def send_message():
```

```
    retrieved_chunks = retrieve_relevant_chunks(
        user_query
    )

    context = build_context(
        retrieved_chunks
    )

    response = generate_response(
        context,
        user_query
    )

    return {
        "response": response
    }
```

This implementation enables researchers to interact naturally with their research collections while maintaining continuity across multiple conversations.

By the completion of this milestone, ResearchPilot AI successfully transformed from a document management platform into a fully AI-powered research assistant capable of intelligent retrieval, reasoning, analysis, and context-aware research support.

Milestone 6: Testing, Optimization, and Deployment

This milestone focused on validating the overall functionality, performance, security, and reliability of the ResearchPilot AI platform. Comprehensive testing was conducted across frontend, backend, database, vector search, and AI components to ensure seamless integration between all modules. Performance optimization techniques were applied to improve response times, retrieval accuracy, and user experience. Finally, the platform was configured for deployment and real-world usage.

Activity 6.1: Running the Backend

The backend services were developed using FastAPI and executed through the Uvicorn ASGI server. The development server supports automatic reloading, allowing code changes to be reflected instantly during development.

```bash
# Navigate to backend directory
cd backend
```

```
# Activate virtual environment
venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Run FastAPI server
uvicorn app.main:app --reload
```

The backend handles:

* User authentication
* Workspace management
* Research paper search
* PDF processing
* Vector embedding generation
* AI tool execution
* Research chat operations

The API server acts as the central communication layer between the frontend, database, vector store, and AI services.

Activity 6.2: Running the Frontend

The frontend application was built using React, TypeScript, and Tailwind CSS. Vite was used as the development build tool to provide fast startup times and hot module replacement during development.

```bash
# Navigate to frontend directory
cd frontend

# Install dependencies
npm install
```

```
# Start development server
npm run dev
```

The frontend provides access to:

* Dashboard
* Search Papers
* Workspace Management
* Upload PDF
* AI Tools Hub
* Research Chat
* Document Space
* Authentication Pages

The user interface communicates with backend APIs through REST endpoints and provides real-time interaction with AI-powered services.

Activity 6.3: Configure Environment Variables and Security Settings

Environment variables were used to securely manage application configuration and API credentials.

```env
# backend/.env
GROQ_API_KEY=your_groq_api_key
SECRET_KEY=your_secret_key

DATABASE_URL=sqlite:///research_pilot.db

CHROMA_DB_PATH=./data/chroma_db
UPLOAD_DIR=./data/uploads
```

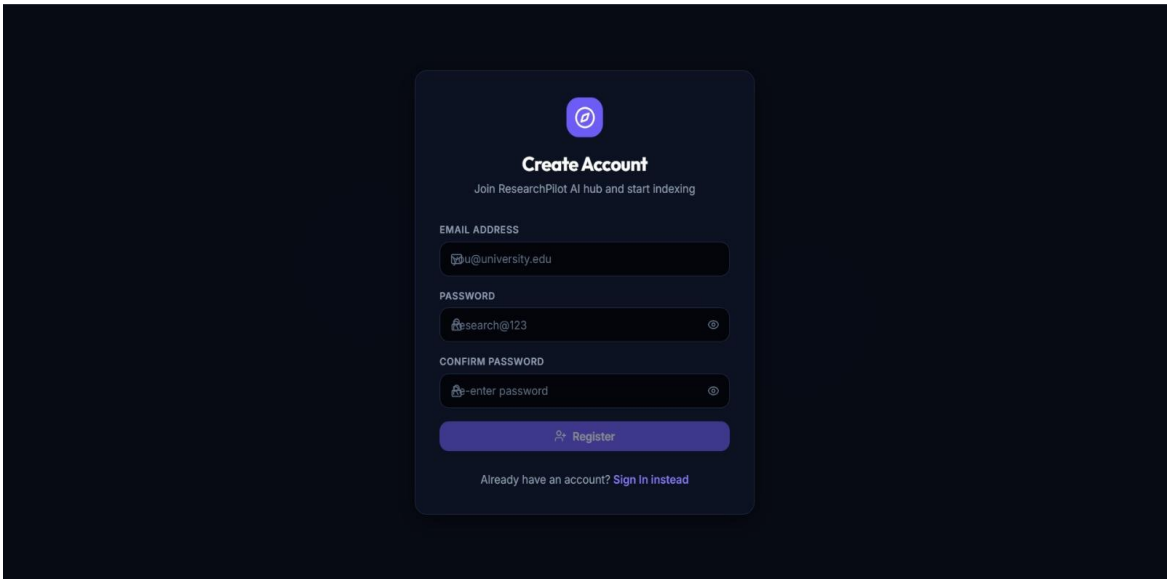
Cross-Origin Resource Sharing (CORS) was configured to allow secure communication between the frontend and backend applications.

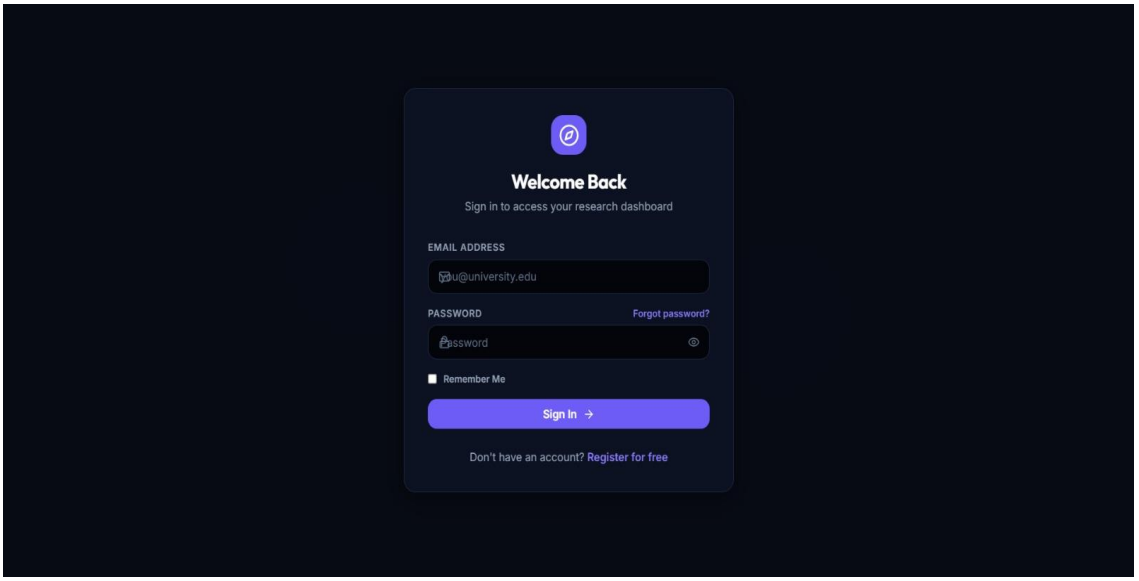
```python
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

These configurations ensure secure API communication while protecting sensitive credentials from exposure.

Login Page

The Login Page serves as the secure entry point to the ResearchPilot AI platform. Users authenticate using their registered credentials and receive JWT access tokens for accessing protected resources. The page includes input validation, secure authentication workflows, and password recovery options.

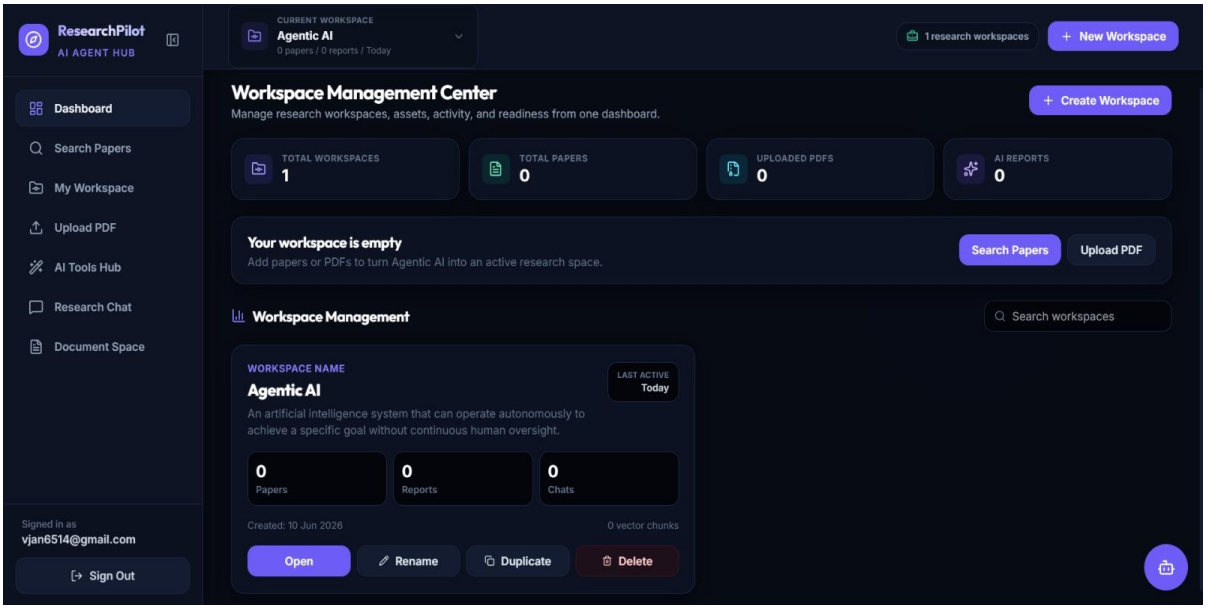




Dashboard:

The Dashboard acts as the central control panel of the platform. It provides an overview of active workspaces, imported research papers, uploaded documents, recent activity, and quick navigation to core platform features.

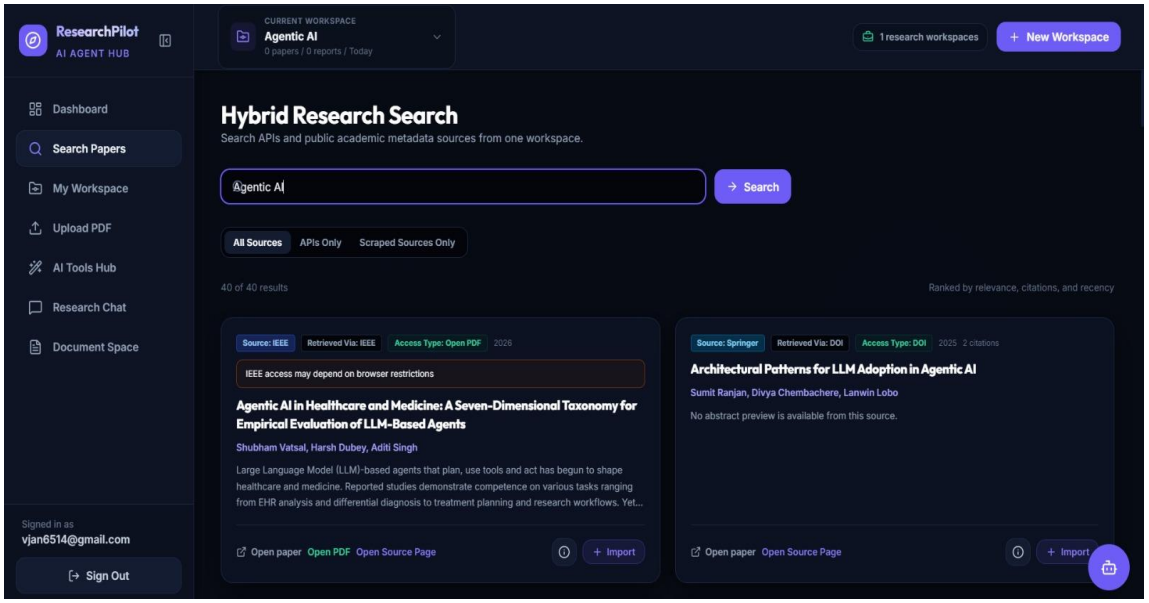
Researchers can monitor their research progress and quickly access frequently used tools from a single location.



Search Papers Page

The Search Papers page enables users to search multiple academic databases including IEEE, arXiv, Semantic Scholar, OpenAlex, CrossRef, ACM Digital Library, DOAJ, and PubMed.

Researchers can enter keywords, browse results, review abstracts, and import relevant papers directly into their workspaces.



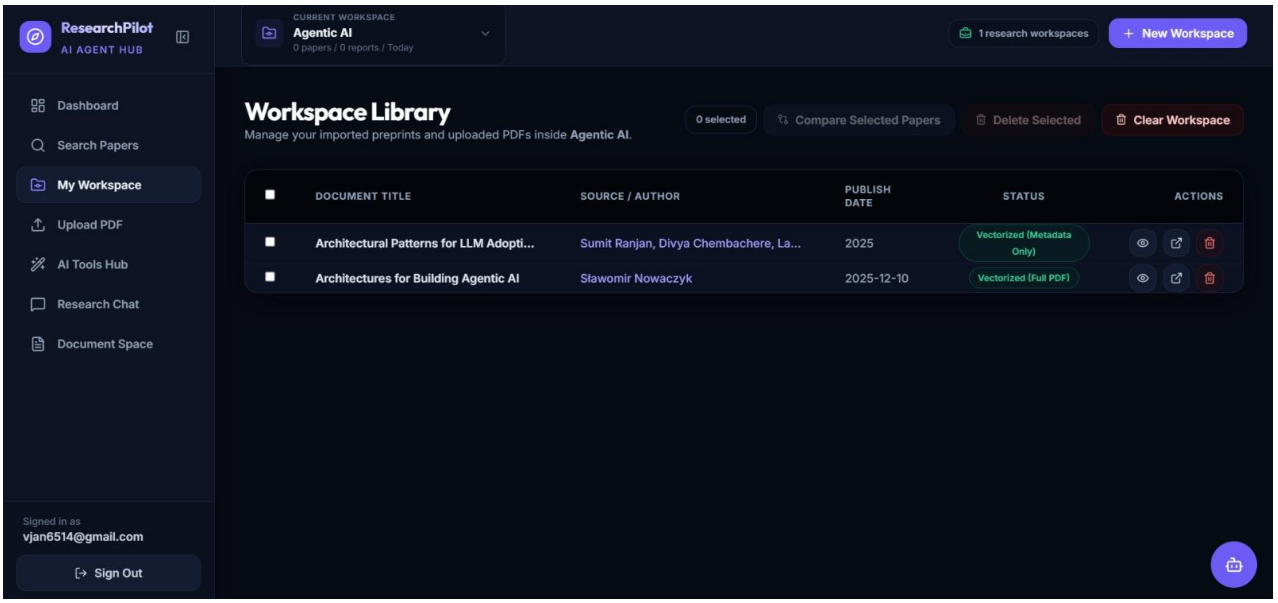
Workspace Management:

Workspaces serve as project-specific research environments where users organize imported papers and uploaded documents.

Each workspace maintains:

* Imported papers
* Uploaded PDFs
* AI-generated reports
* Research chat sessions
* Conversation history

This structure helps researchers manage multiple independent projects efficiently.



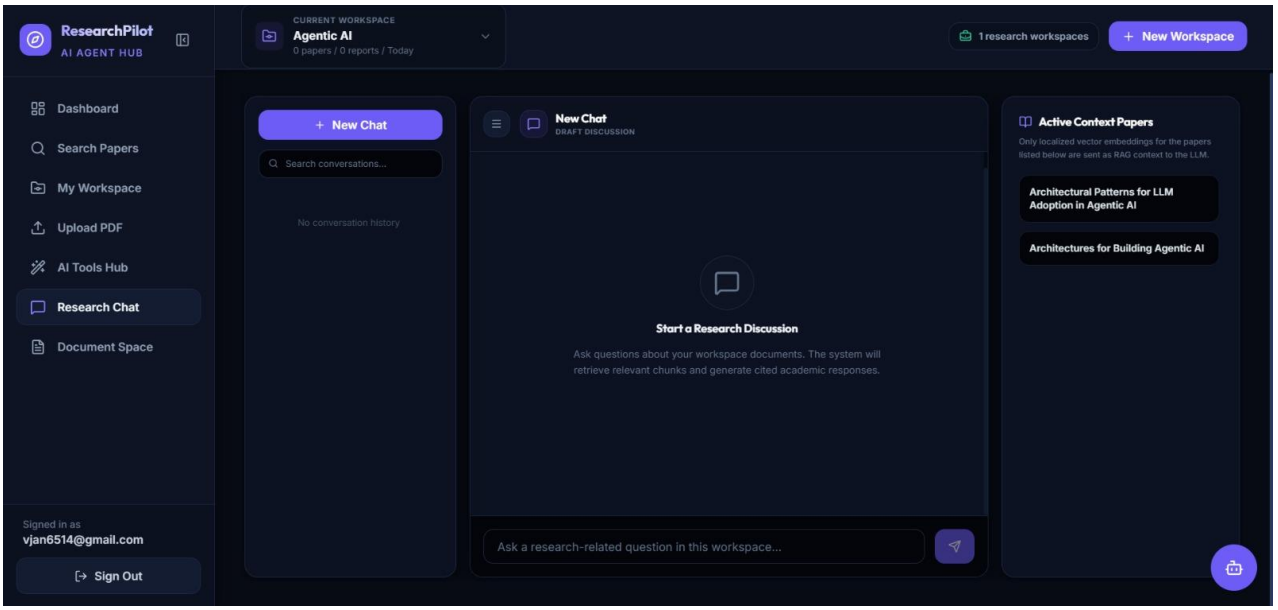
Research Chat

The Research Chat module provides context-aware conversational interaction with uploaded research papers.

Key features include:

* Persistent conversation history
* Multiple chat sessions
* Session pinning and renaming
* Workspace-specific context
* AI-powered question answering

The assistant retrieves relevant document chunks from the vector database before generating responses, ensuring factual and document-grounded answers.



[Insert Research Chat Screenshot Here]

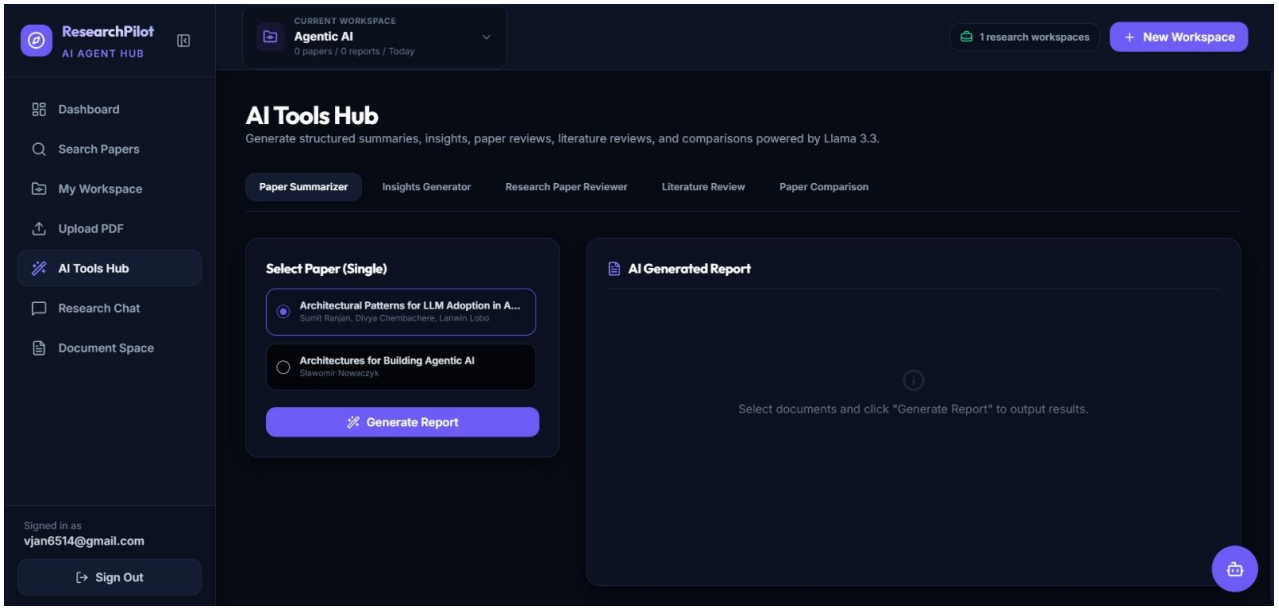
AI Tools Hub

The AI Tools Hub provides advanced research analysis capabilities powered by the Groq Llama 3.3 70B model.

Available tools include:

* Paper Summarizer
* Literature Review Generator
* Paper Comparison
* Insights Generator
* Research Paper Reviewe

These tools automate complex research tasks and significantly reduce manual analysis effort.



Upload PDF:

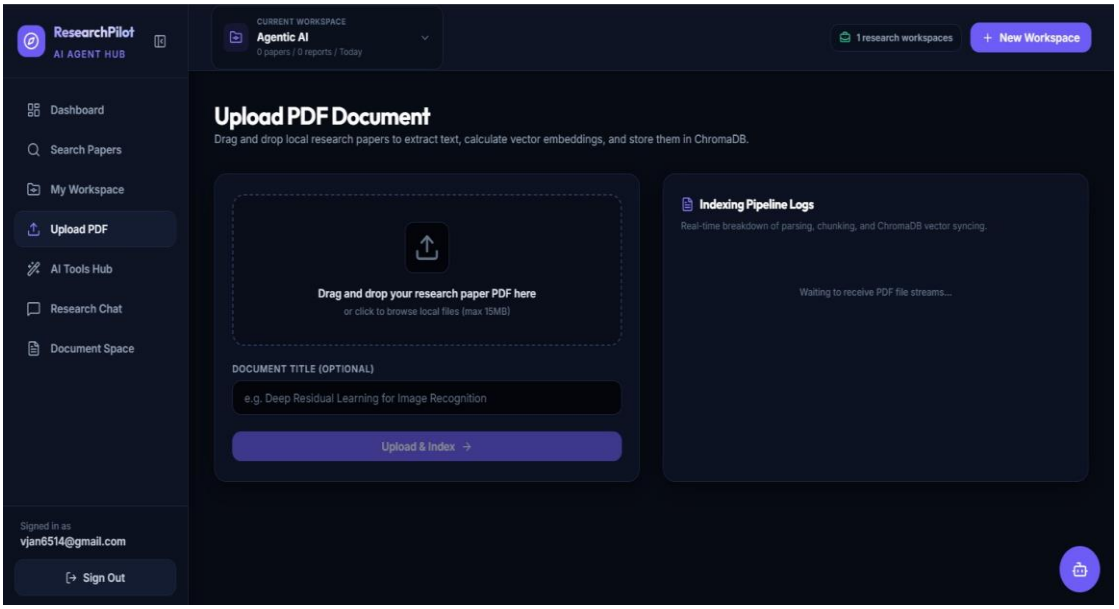
The Upload PDF module allows researchers to add personal research papers directly into the platform.

Uploaded documents undergo:

1. PDF processing
2. Text extraction
3. Chunk generation
4. Vector embedding creation
5. ChromaDB indexing

This process enables semantic search and AI-powered analysis of uploaded content/

**[Insert Upload PDF Screenshot Here]*.



By completing this milestone, ResearchPilot AI was successfully tested, optimized, and prepared for deployment as a complete AI-powered research management and analysis platform

CONCLUSION :

The introduction of research paper management with AI technology, represented by ResearchHub AI, is a major step forward. The platform's intelligent solution, built with the React and TypeScript frontend, the FastAPI backend, and the Groq's Llama 3.3 model, enables researchers to concentrate on insights, not information management, as the platform automates paper data discovery. The agentic AI architecture allows researchers to interact with academic literature and gain meaning from complex research domains in a new way, with the ability to reason autonomously and analyze in a context-aware manner. Basic problems in current research processes are solved by the platform's multi-agent features such as semantic search based on vector embeddings, conversational AI with persistent context and autonomous paper analysis. ResearchHub AI is a prime example of how AI agents can transform knowledge work, simplify repetitive tasks, and enable researchers to gain deeper insights into their research at a massive scale. Key features are independent research support, which can analyze papers and generate a synthesis result, semantics-based search functions, which can search on the basis of concepts, multi-workspace management, which allows parallel research projects, scalable architecture, which supports multiple users.